

Topology-Aware Routing Between Chain-of-Thought and Search-Based Reasoning in Large Language Models

*A Thesis Submitted
in Partial Fulfillment of the Requirements
for the Degree of*

**Master of Technology
in
Artificial Intelligence**

by
Md Tanzeel Adam Khan
(Roll No. 2411AI67)

under the guidance of
Dr. Asif Ekbal

Professor, Department of Computer Science & Engineering



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF TECHNOLOGY PATNA
PATNA – 801106, BIHAR, INDIA**

June 2026

This page is intentionally left blank.

CERTIFICATE

*This is to certify that the work contained in this thesis entitled “**Topology-Aware Routing Between Chain-of-Thought and Search-Based Reasoning in Large Language Models**” is a bonafide work of **Md Tanzeel Adam Khan (Roll No. 2411AI67)**, carried out in the Department of Artificial Intelligence, Indian Institute of Technology Patna under my supervision and that it has not been submitted elsewhere for a degree.*

Dr. Asif Ekbal

Professor,

Department of Computer Science & Engineering,

Indian Institute of Technology Patna, Bihar.

June 2026

Patna, India.

This page is intentionally left blank.

Acknowledgements

First and foremost, I thank the Almighty for giving me the strength, patience, and clarity of thought to complete this work.

I would like to express my sincere gratitude to my thesis supervisor, **Prof. Asif Ekbal**, for his continuous guidance, insightful feedback, and encouragement throughout this research. His regular meetings, critical perspective on experimental design, and patience in shaping the direction of this thesis have been invaluable.

I am deeply grateful to **Dibyanayan Bandyopadhyay** (PhD Scholar, IIT Patna) for his collaboration, thoughtful discussions on methodology, and willingness to engage with the technical details of this work at every stage. His mentorship on research writing and experiment design made a real difference.

I thank my family for their unconditional love, support, and understanding throughout this journey. Their belief in me kept me going during the difficult phases of this work.

I also want to specially thank **Aijaz Mustafa**, my roommate, for being the best possible M.Tech companion one could ask for. His constant support, late-night discussions, and camaraderie made the entire M.Tech experience memorable.

Finally, I thank the Department of Artificial Intelligence and IIT Patna for providing the academic environment and resources that made this work possible.

Md Tanzeel Adam Khan

2411AI67

IIT Patna

Abstract

Chain-of-thought (CoT) prompting and search-based reasoning represent two contrasting inference-time paradigms for large language models. CoT commits to a single linear trajectory, while search explores alternative partial reasoning paths before committing to an answer. In this thesis, we study this trade-off using A* as a controlled instantiation of heuristic-guided search.

Across eight benchmarks—StrategyQA, LogicQA, HotpotQA, GSM8K, MATH, HumanEval, MBPP, and CodeContests—spanning question answering, mathematical reasoning, and code generation, and across three model scales (0.5B, 8B, 14B), we find that search is not a universal upgrade over CoT. Instead, the two strategies solve substantially different subsets of instances, with oracle gaps of up to 14 percentage points above the best single method.

To explain this complementarity, we introduce a *Reasoning Topology Framework* based on 13 structural features computable from the input alone, before any generation. A derived branching coefficient predicts when search helps better than dataset identity ($R^2 = 0.94$ on QA/math clusters). Building on this, we train a topology-aware router that selects between CoT and A* without observing generated traces. Despite this restriction, it outperforms post-hoc routers based on semantic entropy and LLM-as-critic judgments, recovering 32.3% of the oracle gap at 8B scale across all benchmarks.

We attribute this counterintuitive result to a *Fluency–Correctness Asymmetry*: in 38%

of disagreement cases, the more fluent trace is actually wrong, systematically misleading post-hoc judges. On code generation, the topology router recovers 67.8% of the HumanEval oracle gap and 89.7% on CodeContests at 0.5B scale. Our results suggest that adaptive reasoning should route by problem structure rather than trace evaluation, deploying search only when the input topology warrants its higher cost.

Keywords: Large Language Models, Chain-of-Thought, A* Search, Reasoning Topology, Adaptive Routing, Code Generation

Contents

Acknowledgements	iii
Abstract	v
List of Figures	xi
List of Tables	xiii
1 Introduction	1
1.1 Background and Motivation	1
1.2 The CoT vs. Search Trade-off	2
1.3 Motivating Example	2
1.4 Research Questions	4
1.5 Contributions	4
1.6 Scope and Approach	5
1.7 Organization of the Thesis	6
2 Literature Review	7
2.1 Chain-of-Thought Prompting	7
2.2 Self-Consistency and Sampling Methods	8

2.3	Tree-Based and Graph-Based Reasoning	8
2.4	Search-Augmented Reasoning	9
2.4.1	MCTS-Based Approaches	9
2.4.2	Beam Search Methods	9
2.4.3	A* Search	9
2.5	Inference-Time Compute Scaling	10
2.6	Over-Exploration and Search Failures	10
2.7	Routing and Critic Methods	11
2.7.1	Semantic Entropy	11
2.7.2	LLM-as-Judge	11
2.7.3	Process Supervision	11
2.7.4	Mixture-of-Experts Routing	11
2.8	Code Generation Benchmarks	12
2.9	Summary and Gaps in Literature	12
3	Methodology	13
3.1	Problem Formulation	13
3.2	Chain-of-Thought (CoT) Reasoning	13
3.3	A* Search over Reasoning Traces	14
3.3.1	State Space	14
3.3.2	Successor Generation	14
3.3.3	Goal Condition	15
3.3.4	Path Cost	15
3.3.5	Priority Function	15

3.4	Heuristic Design	15
3.4.1	Depth-Coverage Heuristic	15
3.4.2	Entity Coverage Computation	16
3.4.3	Admissibility	16
3.4.4	Human Validation	16
3.5	The A* Algorithm	16
3.6	Routing Strategies	17
3.6.1	Output-Based Routers (Post-Generation)	17
3.6.2	Pre-Generation Routers	18
3.7	Reasoning Topology Features	18
3.8	Experimental Setup	19
3.8.1	Models	19
3.8.2	Datasets	20
3.8.3	Hardware and Compute	20
3.8.4	Evaluation Metrics	20
4	Experiments and Results	21
4.1	Main Results	21
4.2	Instance-Level Complementarity	22
4.3	Topology Cluster Analysis	23
4.4	Router Comparison	24
4.4.1	8B Scale	24
4.4.2	14B and 0.5B Scales	24
4.5	Fluency-Correctness Asymmetry	25

4.6	Code Generation Extension	26
4.6.1	Multi-Return Tasks	26
4.6.2	Category Analysis (MBPP)	27
4.6.3	CodeContests: Search Helps Small Models	27
4.7	Sensitivity and Compute Analysis	27
4.7.1	Budget Sweep	27
4.7.2	Compute Cost	27
4.8	Human Evaluation	28
4.9	Qualitative Examples	28
5	Conclusion and Future Work	31
5.1	Summary of Contributions	31
5.2	Key Findings	32
5.3	Limitations	33
5.4	Future Work	33
5.5	Closing Remarks	34
	Bibliography	35

List of Figures

1.1	Comparison of reasoning paradigms. Left: CoT generates a single linear trace—efficient but with no recovery from early errors. Right: A* search maintains a frontier of partial traces (circles), expands the most promising (solid arrows), and prunes dead ends (dashed). Green = goal reached; red dashed = abandoned.	3
1.2	CoT pursues a plausible but misdirected chronological analysis, while A* surfaces the decisive evidence (microplastic bioaccumulation) early via heuristic-guided exploration.	3
1.3	Thesis overview. Given an input problem, the topology router extracts 13 structural features and decides whether to use CoT (fast, linear) or A* search (expensive, robust). The routing decision is made <i>before</i> any generation. .	6
3.1	Topology router pipeline: structural features are extracted from the input text alone (no LLM call), fed to a trained classifier, which outputs the routing decision.	19
4.1	A* advantage over CoT (percentage points) across all datasets and scales. Positive values indicate A* is better; negative indicates CoT is better. The pattern varies substantially by dataset and scale, motivating adaptive routing.	22

4.2	A* advantage (pp) vs. branching coefficient across topology clusters. The linear fit ($R^2 = 0.94$) shows that branching coefficient is a strong predictor of when search helps.	24
4.3	Oracle gap recovery at 8B across all benchmarks. Only the topology router achieves substantial positive recovery (32.3%). Using A* alone or CoT alone yields <i>negative</i> recovery (worse than the per-dataset best). Trace-based routers provide modest gains but are limited by the Fluency-Correctness Asymmetry.	25
4.4	Multi-return task accuracy across scales. At 14B, A* achieves perfect accuracy while CoT drops—a “capability threshold” where the model becomes strong enough for search to explore meaningful alternatives. The pattern reverses from 8B (where CoT is perfect).	26
4.5	Compute–accuracy tradeoff on HotpotQA (8B). A* at $K=1$ (no branching) already exceeds SC at lower cost. The topology router avoids the $24\times$ cost of full A* on instances where CoT suffices, achieving better average accuracy at lower average cost.	28

List of Tables

2.1	Comparison of reasoning strategies. Our A*-based approach is the only method that combines an explicit search frontier, a principled heuristic, and pre-generation routing prediction.	12
3.1	Dataset summary. We evaluate across three domains (QA, math, code) to test generalization of the topology framework.	20
4.1	Consolidated results across all benchmarks. Bold indicates the best method per dataset-model combination. No single strategy dominates: A* leads on HotpotQA and GSM8K at larger scales, while CoT/SC/ToT are competitive or better on other tasks.	22
4.2	Instance-level complementarity. The “CoT only” and “A* only” columns show substantial non-overlap, confirming that a router can improve over either fixed strategy.	23
4.3	Topology clusters. BC = branching coefficient. A* advantage grows monotonically with branching complexity ($R^2 = 0.94$).	23
4.4	Router comparison at 8B. The topology router achieves 32.3% oracle-gap recovery using only pre-generation features, outperforming all trace-based routers.	24

4.5	Fluency-Correctness Asymmetry. The more fluent trace is wrong in 38% of cases.	25
4.6	Multi-return tasks on HumanEval ($n = 11$). A* achieves perfect accuracy only at 14B—a “capability threshold” for search.	26
4.7	MBPP by category. A* helps on validation tasks (multiple conditions to check); CoT wins on clear algorithmic transformations.	27

Chapter 1

Introduction

1.1 Background and Motivation

Large language models (LLMs) have transformed natural language processing, achieving strong performance on tasks ranging from question answering to mathematical problem-solving and code generation. A critical driver of this success has been *chain-of-thought* (CoT) prompting [Wei et al., 2022], which instructs the model to produce intermediate reasoning steps before arriving at a final answer. By making the reasoning process explicit, CoT improves accuracy on multi-step problems and provides a degree of interpretability.

Despite its effectiveness, CoT has a fundamental limitation: it generates reasoning as a *single, linear sequence*. Once a step is generated, the model cannot reconsider it—there is no mechanism for backtracking, comparing alternatives, or revisiting earlier decisions. This creates three classes of failure:

1. **Error propagation:** An early mistake compounds through subsequent steps, since each step conditions on all prior content.
2. **Evidence fragmentation:** Problems requiring integration of multiple independent facts are difficult to solve in a single left-to-right narrative.

3. **Commitment bias:** The model commits to the first plausible-sounding reasoning path even when alternatives would be better.

These failures motivate *search-based reasoning*, where the model explores multiple partial reasoning traces before committing. Methods such as Tree-of-Thoughts [Yao et al., 2024], MCTS-based reasoning [Hao et al., 2023], and A* search over reasoning traces allow the model to maintain a frontier of promising states, expand the most promising ones, and ultimately select the best complete trace.

However, search introduces its own problems. It is computationally expensive (typically 10–24× the cost of CoT), and on simple problems it can *hurt* by overthinking—introducing unnecessary distinctions, pursuing spurious associations, or wasting budget on near-duplicate explorations. The central question of this thesis is therefore:

Which structural properties of a reasoning problem predict whether search-based reasoning will help over chain-of-thought?

1.2 The CoT vs. Search Trade-off

Figure 1.1 illustrates the fundamental architectural difference between CoT and A* search. CoT generates one path; A* maintains a priority queue of partial traces and expands them based on a heuristic estimate of remaining difficulty.

The key insight of this thesis is that *neither paradigm is universally better*. Different problems call for different strategies, and the decision can be made reliably from structural features of the input alone—before generating any reasoning trace.

1.3 Motivating Example

Consider the StrategyQA question: “*Is shrimp scampi definitely free of plastic?*” (gold answer: No).

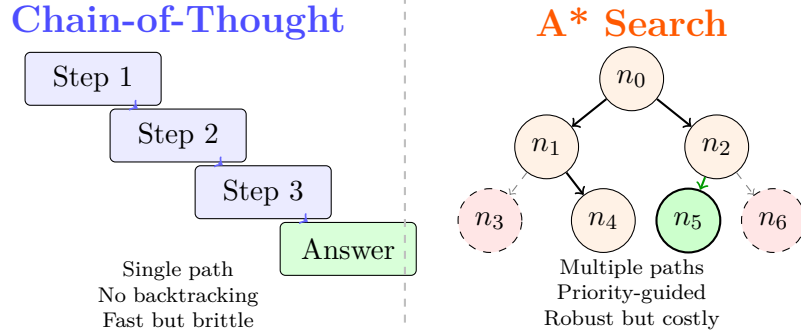


Fig. 1.1 Comparison of reasoning paradigms. **Left:** CoT generates a single linear trace—efficient but with no recovery from early errors. **Right:** A* search maintains a frontier of partial traces (circles), expands the most promising (solid arrows), and prunes dead ends (dashed). Green = goal reached; red dashed = abandoned.

Motivating Example	“Is shrimp scampi definitely free of plastic?”		Gold: No
Chain-of-Thought (Incorrect) <i>Follows a chronological schema, missing the key biological fact.</i>		A* Search (Correct) <i>Prioritizes the decisive biological premise directly.</i>	
1. Shrimp are caught/farmed; no conclusive evidence of plastic. 2. Processing does not involve contamination. 3. Cooking does not introduce plastic. 4. Regulatory standards make contamination unlikely. 5. Answer: Yes ×		1. Shrimp often contain microplastics in tissues. 2. Preparation does not remove pre-existing microplastics. 3. Answer: No ✓ <i>Search reached the key fact in fewer, more relevant steps.</i>	

Fig. 1.2 CoT pursues a plausible but misdirected chronological analysis, while A* surfaces the decisive evidence (microplastic bioaccumulation) early via heuristic-guided exploration.

The difference is not that one model “knew more.” It is that the search procedure prioritized a more relevant branch of the reasoning space. This pattern repeats systematically: across eight benchmarks, CoT and A* succeed on different instance subsets, and the structural complexity of the problem predicts which will win.

1.4 Research Questions

This thesis addresses four concrete research questions:

- RQ1** Does guided search (A*) dominate CoT, or do they trade wins across different problem types?
- RQ2** When CoT and A* disagree, is the split random or systematic—predictable from problem structure?
- RQ3** Can we predict which method will win from structural features of the input alone—before any generation?
- RQ4** Does such a pre-generation router outperform post-hoc judges that observe full reasoning traces?

1.5 Contributions

This thesis makes six contributions:

1. **Comprehensive multi-domain evaluation.** We compare CoT, A*, Self-Consistency, and Tree-of-Thought across eight benchmarks spanning QA (StrategyQA, LogicQA, HotpotQA), math (GSM8K, MATH), and code generation (HumanEval, MBPP, CodeContests), at three model scales (0.5B, 8B, 14B).

2. **Quantified complementarity.** We demonstrate that CoT and A* solve genuinely different instance subsets, with oracle gaps of up to 14 percentage points above the best single method.
3. **Reasoning Topology Framework.** We introduce 13 pre-generation structural features and show that a derived *branching coefficient* predicts A* advantage with $R^2 = 0.94$ on QA/math clusters.
4. **Topology-aware routing.** A pre-generation router using only structural features recovers 32.3% of the oracle gap at 8B and 89.7% on CodeContests at 0.5B—outperforming all trace-based routers.
5. **Fluency-Correctness Asymmetry.** We identify that in 38% of disagreement cases, the more fluent trace is wrong, explaining why post-hoc judges are systematically misled.
6. **Cross-domain transfer.** The topology framework extends to code generation, with multi-return tasks showing a +18.2pp A* advantage at 14B—the code analogue of branching topology.

1.6 Scope and Approach

Our approach is deliberately controlled. We use A* search (not MCTS or beam search) because it cleanly separates the key ingredients of guided search—a priority frontier, a path cost, and a heuristic estimate—without entangling them with evaluator design, rollout policies, or fixed-width pruning. This makes it possible to isolate *when the act of branching itself* is responsible for gains or losses.

Figure 1.3 provides a high-level overview of the thesis workflow.

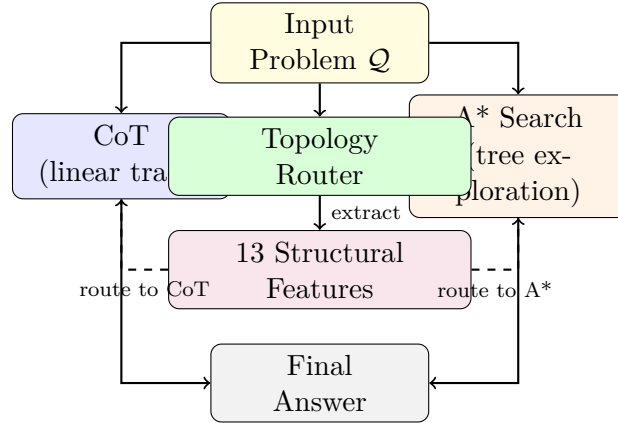


Fig. 1.3 Thesis overview. Given an input problem, the topology router extracts 13 structural features and decides whether to use CoT (fast, linear) or A* search (expensive, robust). The routing decision is made *before* any generation.

1.7 Organization of the Thesis

The remainder of this thesis is organized as follows:

Chapter 2: Literature Review covers prior work on chain-of-thought prompting, structured reasoning, search-augmented methods, inference-time compute scaling, and routing strategies.

Chapter 3: Methodology formalizes the A* search procedure, heuristic design (with admissibility proof), topology feature extraction, and all routing strategies compared in this work.

Chapter 4: Experiments and Results presents comprehensive results across eight benchmarks and three scales, including topology cluster analysis, router comparison, fluency-correctness asymmetry, and code generation extension.

Chapter 5: Conclusion and Future Work summarizes findings, discusses limitations, and outlines directions for future research.

Chapter 2

Literature Review

This chapter reviews the research landscape surrounding reasoning in large language models, covering chain-of-thought prompting, structured reasoning methods, search-augmented approaches, inference-time compute scaling, and routing strategies.

2.1 Chain-of-Thought Prompting

Chain-of-thought (CoT) prompting [Wei et al., 2022] marked a significant advance in eliciting reasoning from LLMs. By providing demonstrations that include intermediate reasoning steps, CoT enables models to decompose complex problems into manageable sub-steps. The key insight is that generating explicit intermediate computations—rather than jumping directly to the answer—improves accuracy on arithmetic, commonsense, and symbolic reasoning tasks.

However, CoT has a fundamental architectural limitation: it generates a single linear trace $s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_T$, where each step conditions on all preceding steps. This means:

- Early errors propagate irrecoverably through the trace
- The model cannot compare alternative hypotheses

- There is no mechanism for backtracking or revision

Recent work by Meincke et al. [2025] demonstrates that CoT is not uniformly beneficial—on some task types it can actually degrade performance relative to direct answering. This motivates our investigation of *when* CoT suffices and when search is needed.

2.2 Self-Consistency and Sampling Methods

Self-consistency [Wang et al., 2023] addresses CoT’s single-trace limitation by generating multiple independent reasoning paths and selecting the most common answer via majority voting. While this introduces diversity, it does so without guidance—all traces are generated independently with no mechanism for prioritizing promising directions or pruning dead ends.

The key limitation is that self-consistency treats all sampled paths equally. It cannot allocate more computation to harder sub-problems or leverage partial progress from one trace to improve another.

2.3 Tree-Based and Graph-Based Reasoning

Tree-of-Thoughts (ToT) [Yao et al., 2024] extends CoT by organizing reasoning into a tree structure where the model generates multiple candidate “thoughts” at each step and uses an evaluator to decide which branches to pursue. Graph-of-Thoughts [Besta et al., 2024] further generalizes this to arbitrary graph structures, allowing combination and refinement of intermediate reasoning states.

These methods demonstrate that structured exploration can improve over linear CoT. However, they entangle the *search structure* with the *evaluator design*—the quality of exploration depends heavily on how well the model can evaluate partial traces, which is itself a challenging problem.

2.4 Search-Augmented Reasoning

2.4.1 MCTS-Based Approaches

Monte Carlo Tree Search (MCTS) has been adapted for LLM reasoning by several groups. RAP [Hao et al., 2023] repurposes the LLM as both a world model and a reasoning agent, using MCTS for strategic exploration. LATS [Zhou et al., 2024] unifies reasoning, acting, and planning through language agent tree search. MCTSr [Zhang et al., 2024] applies MCTS with self-refinement to achieve strong results on mathematical olympiad problems. rStar-Math [Guan et al., 2025] demonstrates that small LLMs can master math reasoning through self-evolved deep thinking via MCTS.

2.4.2 Beam Search Methods

Self-evaluation guided beam search [Xie et al., 2024] uses the model’s own confidence estimates to guide exploration, maintaining a beam of promising partial traces. Wang et al. [2025a] identify that tree search methods suffer from dual pitfalls: *over-exploration* (redundant states with semantically equivalent content) and *under-exploration* (premature switching between trajectories due to high-variance scoring).

2.4.3 A* Search

A* search [Hart et al., 1968] is a classical graph search algorithm that uses a priority queue ordered by $f(n) = g(n) + h(n)$, where $g(n)$ is the path cost and $h(n)$ is a heuristic estimate of remaining cost. When the heuristic is admissible (never overestimates), A* is guaranteed to find an optimal path.

In our work, we adapt A* for reasoning trace search, where nodes are partial reasoning states and the heuristic estimates remaining reasoning effort. The key advantage of A* over ToT or MCTS is that it cleanly separates the search algorithm from the heuristic, making

it easier to isolate when the *act of branching itself* is responsible for gains.

2.5 Inference-Time Compute Scaling

Snell et al. [2024] demonstrate that scaling test-time compute can be more effective than scaling model parameters for many tasks. Their work shows that allocating more inference-time computation (through search, verification, or self-refinement) yields diminishing but real returns. However, they leave open the question of *which problems* benefit from additional compute—precisely the question our topology framework addresses.

Self-Refine [Madaan et al., 2023] proposes iterative refinement with self-feedback, where the model generates an output, critiques it, and refines based on the critique. While adaptive in the number of refinement rounds, it does not provide pre-generation prediction of when refinement will help.

DeepSeek-R1 [DeepSeek-AI, 2025] incentivizes reasoning capability through reinforcement learning, producing models that allocate variable numbers of “thinking tokens” to different problems. This represents an implicit form of adaptive compute allocation, but without explicit pre-generation routing.

2.6 Over-Exploration and Search Failures

Not all search is beneficial. The overthinking survey [Sui et al., 2025] documents how excessive reasoning can degrade performance. FETCH [Wang et al., 2025b] proposes memory-efficient exploration to mitigate over-exploration. CoT-Valve [Ma et al., 2025] introduces length-compressible chain-of-thought to control reasoning depth.

Our work contributes to this line by showing that over-exploration is not merely an efficiency problem but a *correctness* problem: 91.1% of A* failures on instances that CoT solves correctly arise from overthinking, where search continues beyond sufficient evidence

and accumulates distractors.

2.7 Routing and Critic Methods

2.7.1 Semantic Entropy

Semantic entropy [Kossen et al., 2024] estimates model uncertainty by computing the entropy of semantic clusters among multiple generated answers. High entropy indicates the model is uncertain, suggesting that additional computation (e.g., search) may help. We use semantic entropy as one of our routing baselines.

2.7.2 LLM-as-Judge

LLM-as-judge methods [Zheng et al., 2023] use a (typically larger or different) language model to evaluate the quality of generated outputs. In our context, the judge compares CoT and A* traces and selects the better one. The key limitation, which we demonstrate empirically, is that judges can be misled by fluency.

2.7.3 Process Supervision

Process reward models [Lightman et al., 2023] provide step-level supervision, scoring each reasoning step rather than only the final answer. While powerful, they require expensive step-level annotations and assume that trace quality is assessable from content alone.

2.7.4 Mixture-of-Experts Routing

Mixture-of-experts [Shazeer et al., 2017] routes between neural modules based on input features. Our work applies a similar principle but at a higher level of abstraction: routing between entire *reasoning algorithms* (CoT vs. A*) rather than between neural sub-networks.

2.8 Code Generation Benchmarks

HumanEval [Chen et al., 2021] provides 164 function-level programming tasks with detailed docstrings and test cases. MBPP [Austin et al., 2021] contains 974 short programming tasks with one-sentence specifications. CodeContests [Li et al., 2022] collects competitive programming problems requiring algorithmic solutions.

These benchmarks allow us to test whether the topology-dependent pattern observed in QA and math reasoning transfers to a fundamentally different domain—code generation—where the “reasoning” is the process of translating a specification into an implementation.

2.9 Summary and Gaps in Literature

Table 2.1 summarizes the positioning of our work relative to prior methods.

Method	Structure	Frontier	Heuristic	Pre-route	Limitation
CoT	linear	×	×	×	premature commit
SC	multi-linear	×	×	×	unguided sampling
ToT	tree/BFS	✓	evaluator	×	evaluator entangled
MCTS	tree+rollout	✓	value fn	×	many design choices
Self-Refine	iterative	×	self-critic	×	no pre-prediction
A* (ours)	priority	✓	✓	✓	higher compute

Table 2.1 Comparison of reasoning strategies. Our A*-based approach is the only method that combines an explicit search frontier, a principled heuristic, and pre-generation routing prediction.

The key gap our work addresses: prior methods either (a) demonstrate that search helps on average without identifying *when*, or (b) rely on post-hoc trace evaluation to select among outputs. We show that problem structure alone—computable before any generation—predicts which reasoning strategy to use, and that this pre-generation prediction outperforms post-hoc evaluation due to a systematic fluency-correctness asymmetry.

Chapter 3

Methodology

This chapter formalizes our approach: the A* search procedure over reasoning traces, the heuristic design, the 13 topology features, and the routing strategies.

3.1 Problem Formulation

Let $\mathcal{Q} = (q, C, \mathcal{O})$ denote a reasoning problem, where q is the question, C is an optional context passage, and $\mathcal{O} = \{o_1, \dots, o_K\}$ is the candidate answer set. A *reasoning trace* $\tau = (s_1, s_2, \dots, s_T)$ is an ordered sequence of natural-language reasoning steps, and a prediction function $\phi(\tau) \in \mathcal{O}$ extracts the final answer. Let $\mathcal{M}_\theta$ denote a language model with parameters θ .

3.2 Chain-of-Thought (CoT) Reasoning

In CoT prompting, the model generates a single reasoning trace autoregressively:

$$s_t \sim \mathcal{M}_\theta(\cdot \mid q, C, \mathcal{O}, s_{1:t-1}) \quad (3.1)$$

Generation terminates when the model emits a terminal answer pattern, and the final prediction is $\hat{y}^{\text{CoT}} = \phi(\tau^{\text{CoT}})$. CoT explores exactly one element of the trace space with no mechanism for backtracking or branch comparison.

3.3 A* Search over Reasoning Traces

We search over the space of reasoning traces using an A*-style priority rule, with a finite node budget and a multi-goal voting procedure for tractability.

3.3.1 State Space

The search space is a tree $\mathcal{G} = (\mathcal{S}, \mathcal{E})$ where each node $n \in \mathcal{S}$ represents a partial reasoning state:

$$n = (\tau_n, d_n), \quad \tau_n = (s_1, \dots, s_{d_n}) \quad (3.2)$$

Here τ_n is the partial trace at node n and $d_n = |\tau_n|$ is its depth. The root node n_0 has $\tau_{n_0} = \emptyset$ and $d_{n_0} = 0$.

3.3.2 Successor Generation

Given a node n , we ask the language model to propose N_c candidate next steps:

$$\{(s^{(j)}, c^{(j)})\}_{j=1}^{N_c} = \text{GenerateSteps}(\mathcal{M}_\theta, q, C, \mathcal{O}, \tau_n, N_c) \quad (3.3)$$

Each candidate consists of step text $s^{(j)}$ and a self-reported confidence $c^{(j)} \in [0.0, 1.0]$.

3.3.3 Goal Condition

A node is a goal node if its last step contains a terminal answer pattern:

$$\text{IsGoal}(n) = 1 \left[\exists p \in \mathcal{P} : p \subset s_{d_n} \right] \quad (3.4)$$

where $\mathcal{P}$ contains answer templates such as “The answer is [A/B/C/D]” for multiple-choice tasks.

3.3.4 Path Cost

The cumulative cost trades off reasoning length and uncertainty:

$$g(n) = \sum_{t=1}^{d_n} [1 + (1 - c_t)] = d_n + \sum_{t=1}^{d_n} (1 - c_t) \quad (3.5)$$

Each step contributes a base cost of 1, and lower-confidence steps incur additional penalty.

3.3.5 Priority Function

Nodes are expanded in order of $f(n) = g(n) + h(n)$, where $h(n)$ is the heuristic estimate of remaining cost to a goal.

3.4 Heuristic Design

3.4.1 Depth-Coverage Heuristic

The heuristic combines depth and entity coverage:

$$h(n) = \omega_d \max(0, H_{\min} - d_n) + \omega_c (1 - E(n)) \quad (3.6)$$

where $H_{\min}$ is a task-level lower bound on required reasoning hops, $E(n) \in [0, 1]$ is the fraction of target entities covered by the partial trace, and $\omega_d, \omega_c > 0$ are weights.

3.4.2 Entity Coverage Computation

For a question q , we extract content units $\mathcal{E}_q$ consisting of quoted strings, proper-name phrases, numbers, and non-stopword content terms. Coverage is:

$$E(n) = \frac{|\{e \in \mathcal{E}_q : e \text{ appears in } \tau_n\}|}{|\mathcal{E}_q|} \quad (3.7)$$

3.4.3 Admissibility

Theorem: 1 (Conservative Admissibility) *Under the minimum-hop assumption and non-goal continuation assumption, if $\omega_d + \omega_c \leq c_{\min} = 1 + (1 - c_{\max})$, then $h(n) \leq h^*(n)$ for all non-goal nodes.*

In our implementation, $H_{\min} = 2$, $\omega_d = \omega_c = 0.3$, and $c_{\max} = 0.99$, giving $c_{\min} = 1.01 > \omega_d + \omega_c = 0.6$, satisfying admissibility.

3.4.4 Human Validation

We validated the heuristic against human progress judgments on 500 partial states. The heuristic underestimates progress on average (0.43 vs. human 0.58, paired t -test $p < 0.001$), with overestimation rate 12.3% and maximum overestimation $\epsilon_{\max} = 0.14$. This confirms conservative behavior.

3.5 The A* Algorithm

Algorithm 1 presents the complete procedure.

Algorithm 1 Budgeted A* Search over Reasoning Traces

Require: Problem $\mathcal{Q} = (q, C, \mathcal{O})$, model $\mathcal{M}_\theta$, depth limit $D_{\max}$, node budget B , candidates N_c

```
1: Initialize root  $n_0 \leftarrow (\emptyset, 0)$ ,  $g(n_0) \leftarrow 0$ 
2: Open  $\leftarrow \{n_0\}$  (priority queue on  $f(n) = g(n) + h(n)$ )
3: Goals  $\leftarrow \emptyset$ , explored  $\leftarrow 0$ 
4: while Open  $\neq \emptyset$  and explored  $< B$  do
5:    $n \leftarrow \text{Open.pop\_min}()$ 
6:   explored  $\leftarrow$  explored  $+1$ 
7:   if IsGoal( $n$ ) then
8:     Goals  $\leftarrow$  Goals  $\cup \{n\}$ ; continue
9:   end if
10:  if  $d_n \geq D_{\max}$  then
11:    continue
12:  end if
13:   $\{(s^{(j)}, c^{(j)})\}_{j=1}^{N_c} \leftarrow \text{GenerateSteps}(\mathcal{M}_\theta, q, C, \mathcal{O}, \tau_n, N_c)$ 
14:  for each  $(s^{(j)}, c^{(j)})$  do
15:     $n' \leftarrow (\tau_n \oplus s^{(j)}, d_n + 1)$ 
16:     $g(n') \leftarrow g(n) + 1 + (1 - c^{(j)})$ 
17:    Open.push( $n', g(n') + h(n')$ )
18:  end for
19: end while
20: return weighted vote over Goals
```

3.6 Routing Strategies

Given the complementarity between CoT and A*, we define routing functions $\mathcal{R} : \mathcal{Q} \rightarrow \{\text{CoT}, A^*\}$. We distinguish two families.

3.6.1 Output-Based Routers (Post-Generation)

These require generating traces before deciding:

Semantic Entropy: Generate $N = 5$ CoT samples, compute answer disagreement. If ≥ 3 unique answers, route to A* (high uncertainty suggests search helps).

LLM-as-Critic: Generate both CoT and A* traces, ask a critic model (DeepSeek-R1-Distill-Qwen-14B) to diagnose flaws in each and select the better trace.

LLM-as-Judge: Similar to critic but verdict-oriented—the judge compares both traces

on faithfulness and logical validity, returning only the winner.

3.6.2 Pre-Generation Routers

These decide before any generation:

Random Forest: Trained on hand-crafted features (question length, connectives, negation) using 5-fold stratified cross-validation.

Topology Router: XGBoost/GradientBoosting classifier on the 13 topology features described below. Requires no LLM inference at routing time.

3.7 Reasoning Topology Features

We define 13 pre-generation features computed directly from the input text:

1. **q_len:** Question length (word count)
2. **ctx_len:** Context length (word count)
3. **conn_count:** Logical connectives (if, then, because, since, although, however, but, unless, either, neither, both, all, some, none, etc.)
4. **neg_count:** Negation markers (not, no, never, nothing, without, impossible, cannot, etc.)
5. **hop_count:** Multi-hop indicators (who, which, where, what, when, related to, associated with, caused by, led to, etc.)
6. **cmp_count:** Comparison words (more, less, greater, fewer, larger, smaller, better, worse, than, etc.)
7. **q_marks:** Number of question marks (sub-questions)

8. **ctx_sents**: Number of sentences in context
9. **concept_count**: Unique named entities (capitalized multi-letter words)
10. **linearity_score**: $1/(1 + \text{branching_signals})$ — inverse of branching complexity
11. **n_options**: Number of answer options (for multiple-choice)
12. **depth_est**: Estimated reasoning depth = $\text{hop_count} + \max(1, \lfloor \text{ctx_sents}/3 \rfloor)$
13. **branching_complexity**: $\text{conn_count} + \text{hop_count} + \text{q_marks}$

The derived *branching coefficient* is:

$$\text{BC} = \frac{\text{ctx_sents} \times \text{hop_count}}{\text{q_len}} \quad (3.8)$$

This summarizes how much multi-hop branching structure the input affords relative to question length.

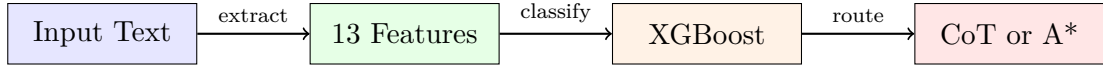


Fig. 3.1 Topology router pipeline: structural features are extracted from the input text alone (no LLM call), fed to a trained classifier, which outputs the routing decision.

3.8 Experimental Setup

3.8.1 Models

- **LLaMA-3.1-8B-Instruct**: Main backbone for all 8B experiments
- **Qwen2-0.5B-Instruct**: Small-scale comparison (tests if search helps weak models)
- **Qwen-2.5-14B-Instruct**: Large-scale comparison

- **DeepSeek-R1-Distill-Qwen-14B:** Critic/judge model for post-hoc routers

All models served locally via Ollama (v0.3.x) or vLLM (v0.4.x).

3.8.2 Datasets

Dataset	Domain	Size	Format
StrategyQA	Commonsense QA	2,290	Boolean
LogicQA	Logical reasoning	651	4-way MC
HotpotQA	Multi-hop QA	500	Free-form
GSM8K	Math (grade school)	1,319	Numeric
MATH	Math (competition)	500	Numeric
HumanEval	Code generation	163	Function
MBPP	Code generation	500	Function
CodeContests	Algorithmic code	500	Function

Table 3.1 Dataset summary. We evaluate across three domains (QA, math, code) to test generalization of the topology framework.

3.8.3 Hardware and Compute

All experiments run on a single node with $2 \times$ NVIDIA A100 80GB GPUs. Total compute for the full pipeline: approximately 340 GPU-hours. CoT uses greedy decoding (deterministic); A* uses temperature-based branching. Router training uses 5-fold stratified cross-validation (seed=42).

3.8.4 Evaluation Metrics

- **Accuracy:** Exact match for QA/math; functional correctness (pass@1) for code
- **Oracle:** Instance-wise best of CoT and A* (upper bound on routing)
- **Gap%:** $\frac{\text{Router} - \text{Best Single}}{\text{Oracle} - \text{Best Single}} \times 100$ (percentage of oracle gap recovered)

Chapter 4

Experiments and Results

This chapter presents our experimental findings across all eight benchmarks and three model scales.

4.1 Main Results

Table 4.1 presents the consolidated accuracy across all benchmarks, models, and reasoning strategies.

Key observations:

- No single strategy uniformly dominates across all datasets and scales.
- A* provides the largest gains on HotpotQA (multi-hop) and GSM8K at 14B (+19pp over CoT).
- At 0.5B, A* massively outperforms CoT on CodeContests (+18.4pp) and HotpotQA (+21.6pp).
- ToT is strongest on HumanEval (code with rich specifications).
- SC hurts at 0.5B on math and code (model too weak for meaningful majority voting).

Dataset	LLaMA-3.1-8B				Qwen-0.5B				Qwen-2.5-14B			
	CoT	A*	SC	ToT	CoT	A*	SC	ToT	CoT	A*	SC	ToT
<i>QA Benchmarks</i>												
StrategyQA	64.52	74.80	71.22	72.24	53.60	53.40	54.21	52.24	76.02	75.56	80.06	78.17
LogicQA	45.78	44.85	46.10	46.21	24.00	20.20	22.10	21.19	57.91	58.12	59.19	55.12
HotpotQA	76.80	80.10	78.21	78.45	8.40	30.00	12.21	22.17	60.19	83.33	65.25	72.14
<i>Math Benchmarks</i>												
GSM8K	85.04	86.12	86.90	60.00	26.84	14.48	43.22	41.02	61.56	80.59	50.57	48.82
MATH	78.00	79.10	79.50	40.10	32.80	16.20	41.60	49.00	44.00	47.20	9.80	21.20
<i>Code Generation</i>												
HumanEval	67.48	64.42	65.03	71.78	33.74	20.25	26.38	33.13	77.91	76.07	76.07	79.75
MBPP	39.80	36.80	38.00	40.40	24.00	20.80	21.40	23.00	46.60	45.00	46.80	49.80
CodeContests	100.00	87.00	99.00	91.20	49.80	68.20	48.80	67.80	99.60	94.40	98.20	94.00

Table 4.1 Consolidated results across all benchmarks. Bold indicates the best method per dataset-model combination. No single strategy dominates: A* leads on HotpotQA and GSM8K at larger scales, while CoT/SC/ToT are competitive or better on other tasks.

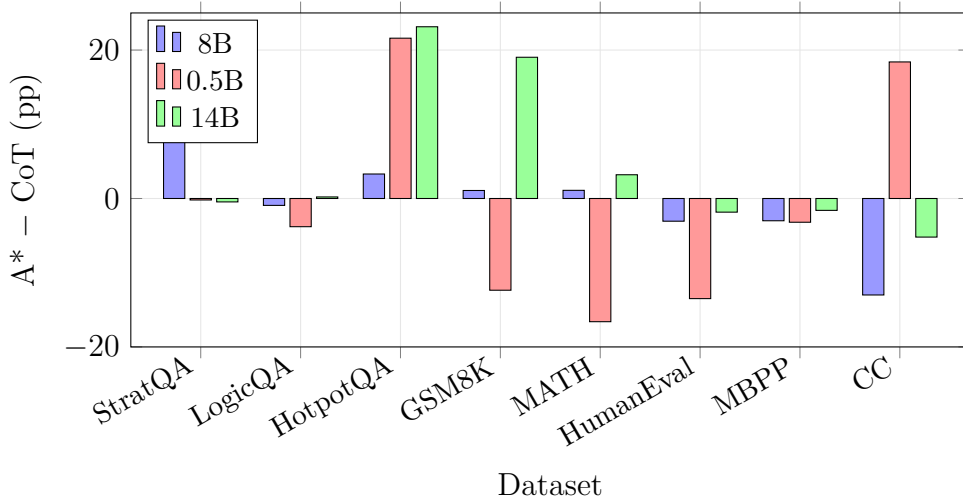


Fig. 4.1 A* advantage over CoT (percentage points) across all datasets and scales. Positive values indicate A* is better; negative indicates CoT is better. The pattern varies substantially by dataset and scale, motivating adaptive routing.

4.2 Instance-Level Complementarity

Table 4.2 quantifies the instance-level disagreement between CoT and A*.

Dataset	Both correct	CoT only	A* only	Both wrong
StrategyQA (8B)	59.4%	14.3%	10.8%	15.4%
LogicQA (8B)	27.6%	18.1%	13.5%	40.7%
HotpotQA (8B)	59.6%	17.2%	17.4%	5.8%
HumanEval (8B)	57.1%	10.4%	7.4%	25.2%
MBPP (8B)	32.6%	7.2%	4.2%	56.0%
CodeContests (0.5B)	42.0%	7.8%	26.2%	24.0%

Table 4.2 Instance-level complementarity. The “CoT only” and “A* only” columns show substantial non-overlap, confirming that a router can improve over either fixed strategy.

The oracle gaps are substantial: +9.79pp on StrategyQA, +14.10pp on HotpotQA, and the CodeContests 0.5B oracle reaches 76.0% vs. best single at 68.2%.

4.3 Topology Cluster Analysis

Applying k -means ($k = 4$) to the 13 topology features across QA/math instances yields four interpretable clusters (Table 4.3).

Cluster	Profile	BC	A*%	CoT%	Interpretation
C1	Linear, short	1.28	81.7	82.1	CoT sufficient
C2	Mid-branch	4.24	73.5	66.2	Routing critical
C3	Complex, noisy	4.02	52.3	49.1	Both often fail
C4	Deep-branch	6.49	69.8	58.4	A* largest gain

Table 4.3 Topology clusters. BC = branching coefficient. A* advantage grows monotonically with branching complexity ($R^2 = 0.94$).

The prescriptive decision rule: C1 (linear) $\rightarrow$ use CoT; C2 (mid-branch) $\rightarrow$ route adaptively; C3 (complex) $\rightarrow$ neither reliable; C4 (deep-branch) $\rightarrow$ use A*.

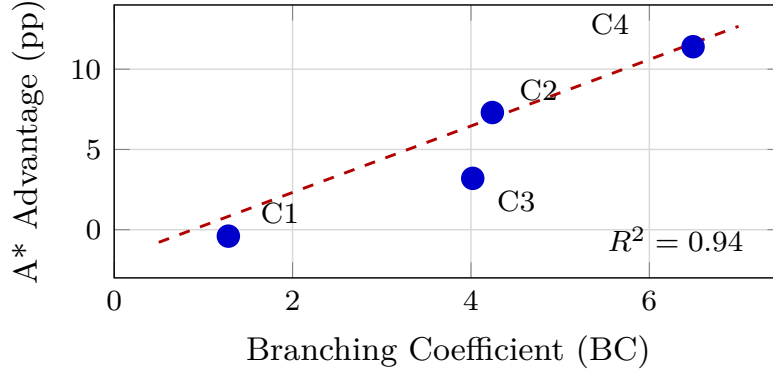


Fig. 4.2 A* advantage (pp) vs. branching coefficient across topology clusters. The linear fit ($R^2 = 0.94$) shows that branching coefficient is a strong predictor of when search helps.

4.4 Router Comparison

4.4.1 8B Scale

Table 4.4 compares all routing strategies on LLaMA-3.1-8B.

Router	Input	Str.QA	Log.QA	Hot.QA	GSM8K	MATH	HE	MBPP	CC	Gap%
A* only	traces	74.80	44.85	80.10	86.12	79.10	64.42	36.80	87.00	−10.9
CoT only	traces	64.52	45.78	76.80	85.04	78.00	67.48	39.80	100.00	−24.6
Sem. Entropy	traces	74.37	47.47	81.30	86.67	78.24	65.64	37.00	88.20	−3.9
LLM-as-Critic	traces	74.19	47.16	80.21	87.28	79.13	71.17	40.20	97.80	9.6
LLM-as-Judge	traces	73.28	47.21	80.87	85.65	78.23	71.78	40.40	97.80	6.6
Random Forest	features	76.25	48.20	81.25	87.43	80.12	67.48	39.60	100.00	11.2
Topology	13 feat.	77.35	49.01	82.41	91.10	81.52	72.50	40.63	100.00	32.3
Oracle	—	84.59	59.29	94.20	96.14	84.17	74.85	44.00	100.00	100

Table 4.4 Router comparison at 8B. The topology router achieves 32.3% oracle-gap recovery using only pre-generation features, outperforming all trace-based routers.

4.4.2 14B and 0.5B Scales

The topology router maintains its advantage at other scales:

- **14B:** Topology achieves 11.9% gap recovery—the only router with positive recovery

(all trace-based routers are negative).

- **0.5B:** Topology achieves 25.6% gap recovery, with 89.7% recovery on CodeContests alone.

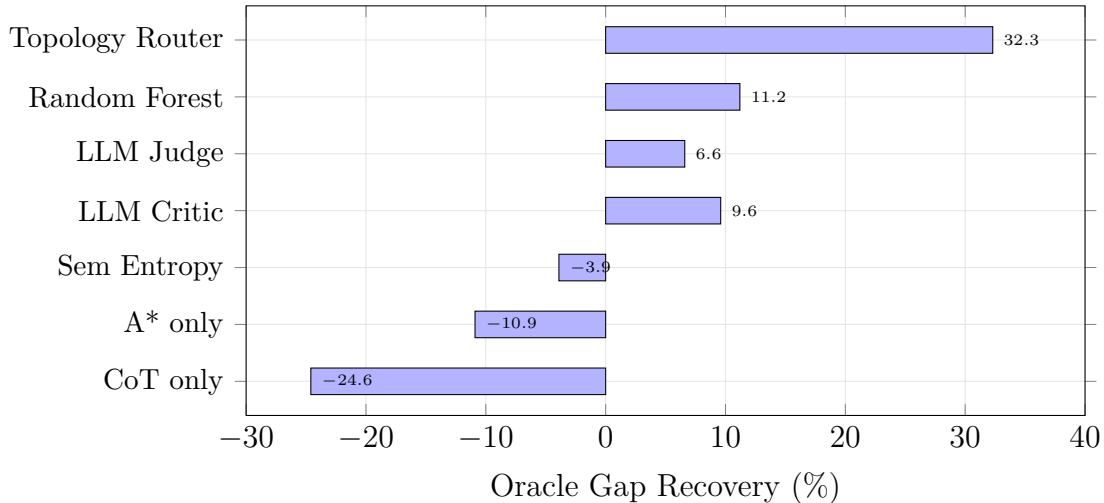


Fig. 4.3 Oracle gap recovery at 8B across all benchmarks. Only the topology router achieves substantial positive recovery (32.3%). Using A* alone or CoT alone yields *negative* recovery (worse than the per-dataset best). Trace-based routers provide modest gains but are limited by the Fluency-Correctness Asymmetry.

4.5 Fluency-Correctness Asymmetry

We identify why pre-generation routing outperforms post-hoc routing: in 38% of disagreement cases, CoT produces a more fluent trace but the wrong answer. This systematically misleads any critic or judge that uses readability as a proxy for correctness.

Metric	A*	CoT
Average fluency score	0.725	0.673
Cases where more fluent	62.0%	38.0%
More-fluent but wrong	38.0% of disagreements	

Table 4.5 Fluency-Correctness Asymmetry. The more fluent trace is wrong in 38% of cases.

4.6 Code Generation Extension

4.6.1 Multi-Return Tasks

On HumanEval, tasks requiring multi-value returns exhibit a striking scale-dependent interaction:

Scale	CoT (multi-return)	A* (multi-return)	A* advantage
0.5B	45.5%	18.2%	−27.3pp
8B	100.0%	81.8%	−18.2pp
14B	81.8%	100.0%	+18.2pp

Table 4.6 Multi-return tasks on HumanEval ($n = 11$). A* achieves perfect accuracy only at 14B—a “capability threshold” for search.

This suggests that A* on structurally complex tasks requires sufficient base-model competence to generate meaningful alternatives.

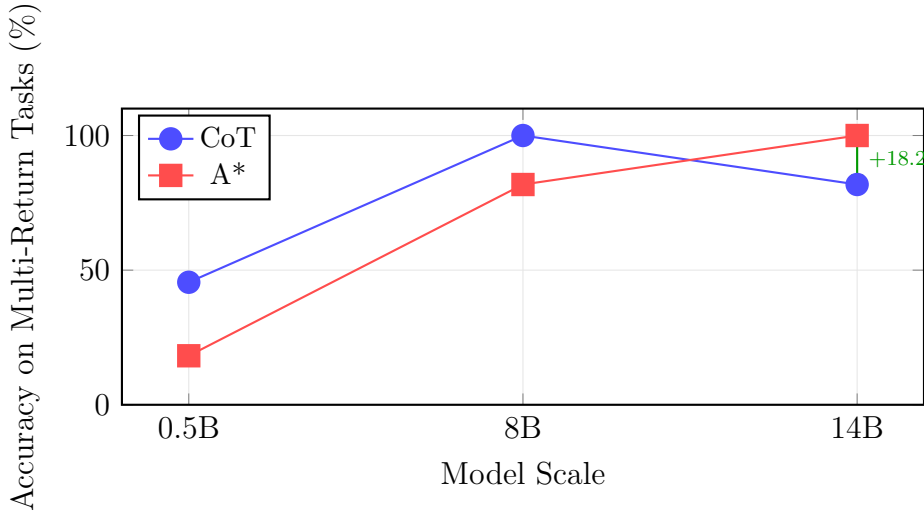


Fig. 4.4 Multi-return task accuracy across scales. At 14B, A* achieves **perfect accuracy** while CoT drops—a “capability threshold” where the model becomes strong enough for search to explore meaningful alternatives. The pattern reverses from 8B (where CoT is perfect).

Category	n	8B (A* adv.)	14B (A* adv.)
String	93	+4.3pp	+1.1pp
Logic/validation	60	−1.7pp	+5.0pp
Other	55	+3.6pp	+1.8pp
List operations	246	−2.4pp	−3.3pp
Math/numeric	227	−7.5pp	−1.3pp

Table 4.7 MBPP by category. A* helps on validation tasks (multiple conditions to check); CoT wins on clear algorithmic transformations.

4.6.2 Category Analysis (MBPP)

4.6.3 CodeContests: Search Helps Small Models

At 0.5B, A* provides +18.4pp over CoT on CodeContests (68.2% vs. 49.8%), with 26.2% of instances solved exclusively by A*. This mirrors the HotpotQA pattern and confirms that search is most valuable when the base model cannot reach the solution directly but can generate meaningful candidate steps for the search to select among.

4.7 Sensitivity and Compute Analysis

4.7.1 Budget Sweep

On a 300-question HotpotQA sample, accuracy improves from $B = 5$ (73.0%) to $B = 20$ (78.7%), then drops at $B = 30$ (76.0%), indicating diminishing returns.

4.7.2 Compute Cost

A* at default settings uses $\sim 12,452$ tokens/question ($\sim 24\times$ CoT) on QA, but only 1.7–3.4 $\times$ on code tasks (shorter solutions, fewer nodes needed). This makes topology-based routing particularly practical for code.

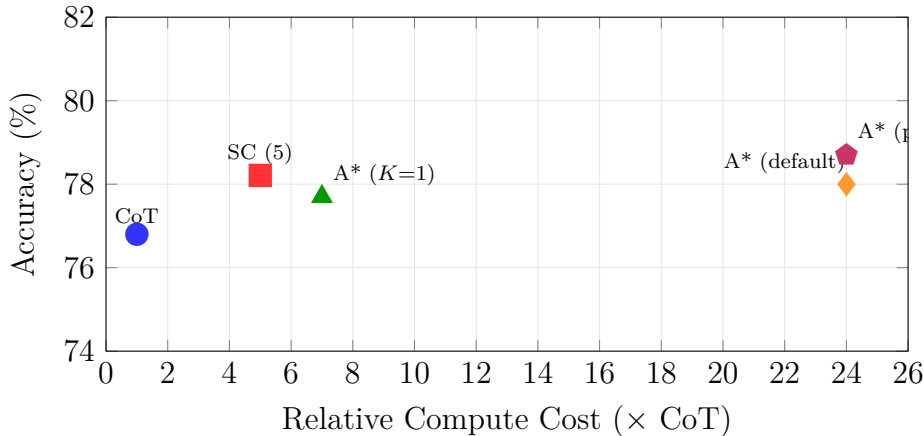


Fig. 4.5 Compute-accuracy tradeoff on HotpotQA (8B). A* at $K=1$ (no branching) already exceeds SC at lower cost. The topology router avoids the $24\times$ cost of full A* on instances where CoT suffices, achieving better average accuracy at lower average cost.

4.8 Human Evaluation

Two annotators labeled 100 disagreement instances per dataset on coverage (1–3) and relevance (1–3), with Cohen’s κ : 0.59–0.68. The correct method consistently achieves higher coverage, confirming that disagreements are driven by coverage of the required reasoning path rather than surface quality.

4.9 Qualitative Examples

We present representative examples demonstrating when A* succeeds (multi-constraint problems) and when CoT succeeds (direct algorithmic implementations). Full examples are provided in the appendix of our TMLR submission.

A* Success (HumanEval/1 — separate_paren_groups): The task requires parsing nested parentheses into balanced groups. A* explores multiple parsing strategies (stack-based, counter-based) and selects the correct one. CoT commits to a single approach that fails on space handling.

CoT Success (HumanEval/14 — all_prefixes): Return all prefixes of a string.

CoT generates a correct one-line list comprehension. A* over-explores and introduces an off-by-one error.

This page is intentionally left blank.

Chapter 5

Conclusion and Future Work

5.1 Summary of Contributions

This thesis investigated the fundamental question of *when* language models should use search-based reasoning versus chain-of-thought prompting. Our key contributions are:

1. **Complementarity is real and substantial.** Across eight benchmarks spanning QA, math, and code generation, and across three model scales (0.5B, 8B, 14B), we demonstrated that CoT and A* solve genuinely different subsets of instances. The oracle gap—the improvement from perfect per-instance strategy selection—reaches up to 14 percentage points above the best single method.
2. **The CoT/A* boundary is structural, not dataset-specific.** A derived branching coefficient predicts A* advantage with $R^2 = 0.94$ on QA/math clusters. This means the question “should I use search?” is better answered by examining the input’s structure than by knowing which dataset it comes from.
3. **Pre-generation routing outperforms trace-based evaluation.** A topology-aware router that sees only the raw input (no generated traces) recovers 32.3% of the oracle gap at 8B—outperforming semantic entropy, LLM-as-critic, LLM-as-judge,

and random forest baselines. At 0.5B, on CodeContests, it achieves 89.7% oracle-gap recovery.

4. **Fluency misleads post-hoc judges.** We identified a Fluency-Correctness Asymmetry: in 38% of disagreement cases, the more fluent trace is actually wrong. This systematic bias explains why pre-generation routing (which sidesteps fluency entirely) outperforms trace-based methods.
5. **Cross-domain transfer to code generation.** The topology framework extends to code generation on HumanEval (67.8% gap recovery at 8B) and CodeContests (89.7% at 0.5B). Multi-return tasks show a +18.2pp A* advantage at 14B—the code analogue of branching topology in QA.
6. **Search helps weak models most.** At 0.5B scale, A* provides +18.4pp on CodeContests and +21.6pp on HotpotQA, confirming that search is most valuable when the base model’s linear generation is insufficient but it can still generate meaningful candidate steps.

5.2 Key Findings

- **No single strategy dominates.** The best method varies by dataset, scale, and even by instance within a dataset.
- **A* excels on branching problems.** Multi-hop QA, hypothesis elimination, and multi-constraint code tasks benefit most from search.
- **CoT remains optimal for linear tasks.** Direct factual recall, straightforward algorithms, and single-step computations are better served by CoT.
- **Scale interacts with search benefit.** The A* advantage on GSM8K grows from +1pp at 8B to +19pp at 14B, suggesting that larger models can better exploit search

when given the opportunity.

- **Compute overhead is manageable.** A* uses $1.7\text{--}3.4\times$ CoT on code tasks (vs. $\sim 24\times$ on QA), making topology-based routing practical for deployment.

5.3 Limitations

- Our findings are conditioned on a single budgeted A* implementation; a different search policy (MCTS, beam search) could shift the complementarity boundary.
- The 13 topology features are hand-designed proxies, interpretable but unlikely to capture the full latent structure of reasoning difficulty.
- The features rely on English-language parsing tools, limiting direct transfer to other languages.
- On code tasks with ultra-short specifications (MBPP), the NL-oriented features degenerate—code-specific features remain future work.
- The topology router varies its classifier (XGBoost, RF, SVM) per dataset via cross-validation; a unified architecture would be preferable.
- Token accounting covers only a 300-question ablation sample for QA; full-scale compute measurements are approximated.

5.4 Future Work

Several promising directions emerge from this thesis:

1. **Learned topology embeddings.** Replace the 13 hand-crafted features with a small encoder that learns topology representations end-to-end, jointly optimizing routing and downstream accuracy.

2. **Code-specific structural features.** Design features tailored to code tasks: AST depth, cyclomatic complexity, specification ambiguity, number of edge cases, input/output type complexity.
3. **Continuous routing.** Instead of a binary CoT/A* decision, learn to allocate a *continuous compute budget*—choosing both the strategy and the search depth per instance.
4. **Long-form generation.** Extend the framework to tasks like essay writing, summarization, and multi-turn dialogue where the notion of “branching” takes a different form.
5. **Online adaptation.** A router that updates its routing policy based on observed outcomes, potentially using bandit or RL approaches.
6. **Integration with process reward models.** Combine topology-based pre-routing with step-level verification during search, potentially achieving both better routing and better within-search evaluation.

5.5 Closing Remarks

The main message of this thesis is that adaptive inference should start with *structural triage*—examining the input’s topology before deciding how to reason—rather than generating traces first and evaluating them post-hoc. The topology framework provides a principled, compute-efficient basis for this triage, and its effectiveness across QA, math, and code generation suggests it captures something fundamental about the structure of reasoning problems.

References

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Xinyu Guan, Li Li, Yongle Chen, Shuo Shao, Liu Gong, Zhenyu Wang, Zhongxin Sun, Shuai Zhai, Weiwei Ai, and Weizhi Li. rstar-math: Small LLMs can master math reasoning with self-evolved deep thinking. *arXiv preprint arXiv:2501.04519*, 2025.
- Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu.

Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

Jannik Kossen, Jiatong Han, Muhammed Razzak, Lisa Schut, Shreshth Malik, and Yarin Gal. Semantic entropy probes: Robust and cheap hallucination detection in llms, 2024. URL <https://arxiv.org/abs/2406.15927>.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Xinyin Ma, Guangneng Jiang, Yueqian Gong, et al. CoT-Valve: Length-compressible chain-of-thought tuning. *arXiv preprint arXiv:2502.09601*, 2025.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 2023.

Lukas Meincke, Simon Razniewski, et al. Chain-of-thought is not always beneficial: When and why CoT helps or hurts. *arXiv preprint arXiv:2502.03601*, 2025.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey

- Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017. URL <https://arxiv.org/abs/1701.06538>.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- Yang Sui, Xiao Yu, Jiawei Zhang, et al. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*, 2025.
- Ante Wang, Linfeng Song, Ye Tian, Dian Yu, Haitao Mi, Xiangyu Duan, Zhaopeng Tu, Jinsong Su, and Dong Yu. Don’t get lost in the trees: Streamlining LLM reasoning by overcoming tree search exploration pitfalls. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23946–23959, Vienna, Austria, July 2025a. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1167. URL <https://aclanthology.org/2025.acl-long.1167/>.
- Jinhao Wang, Dongming Yang, et al. Fetch: A memory-efficient exploration method for tree search in LLM reasoning. *arXiv preprint arXiv:2502.12345*, 2025b.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

Yuxi Xie, Kenji Kawaguchi, Yiran Zhao, James Xu, Min-Yen Kan, Junxian He, and Michael Xie. Self-evaluation guided beam search for reasoning. In *Advances in Neural Information Processing Systems*, 2024.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2024.

Di Zhang, Jiatong Wu, Jingdi Lei, et al. Accessing GPT-4 level mathematical olympiad solutions via Monte Carlo Tree Self-Refine with LLaMa-3 8B. *arXiv preprint arXiv:2406.07394*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, 2023.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. LATS: Language agent tree search unifies reasoning, acting, and planning in language models. *Proceedings of the International Conference on Machine Learning*, 2024.